



INSTITUTO POLITECNICO NACIONAL  
Unidad profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas

---

# APLICACIONES DISTRIBUIDAS

## Servicios ResBul usando NodeJS

---

Palacios Mendoza Jireth Sinai  
Boleta 2023640037

4TM3

Febrero 2026

# Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Objetivo</b>                                  | <b>2</b>  |
| <b>2</b> | <b>Introducción</b>                              | <b>2</b>  |
| <b>3</b> | <b>Desarrollo de los Servicios Web en NodeJS</b> | <b>3</b>  |
| 3.1      | Paso 1: Creación del Proyecto                    | 3         |
| 3.2      | Paso 2: Creación del Servidor                    | 4         |
| 3.3      | Descripción General                              | 9         |
| <b>4</b> | <b>Implementación de los Servicios</b>           | <b>9</b>  |
| 4.1      | Servicio: mascaracteres                          | 9         |
| 4.2      | Servicio: menoscaracteres                        | 10        |
| 4.3      | Servicio: numcaracteres                          | 11        |
| 4.4      | Servicio: palindroma                             | 11        |
| 4.5      | Servicio: concat                                 | 12        |
| 4.6      | Servicio: applysha256                            | 13        |
| 4.7      | Servicio: verifysha256                           | 13        |
| <b>5</b> | <b>Conclusiones</b>                              | <b>14</b> |

## **1. Objetivo**

## **2. Introducción**

### 3. Desarrollo de los Servicios Web en NodeJS

En esta práctica desarrollé una serie de servicios web tipo REST utilizando NodeJS y Express. Todos los servicios fueron diseñados para recibir parámetros en formato JSON y responder también en formato JSON, cumpliendo con los requerimientos establecidos.

#### 3.1. Paso 1: Creación del Proyecto

Primero creé una carpeta para el proyecto llamada:

```
servicios-rest-node
```

Posteriormente inicialicé el proyecto utilizando el siguiente comando:

```
npm init -y
```

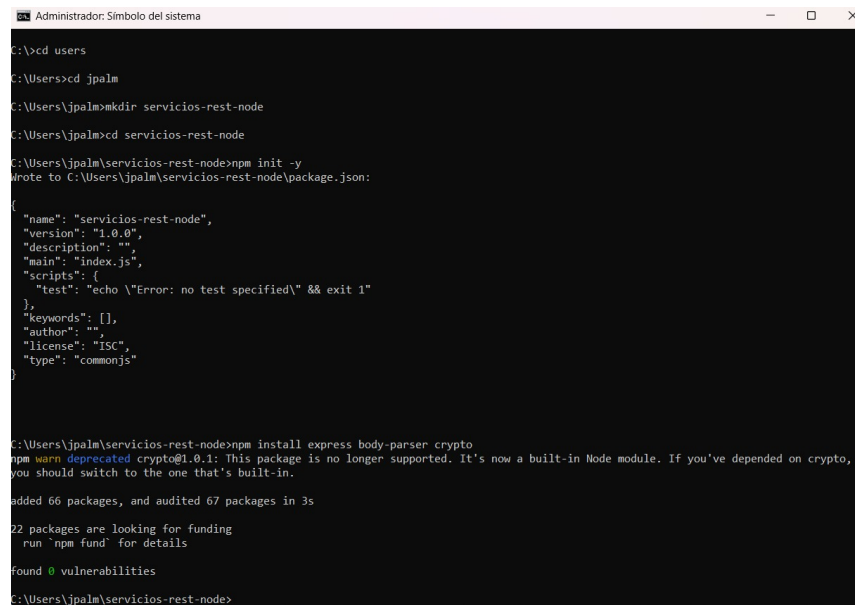
Con esto generé automáticamente el archivo `package.json`.

Después instalé las dependencias necesarias para el desarrollo del servidor:

```
npm install express body-parser crypto
```

Donde:

- **express**: permite crear el servidor web.
- **body-parser**: permite recibir datos en formato JSON.
- **crypto**: se utiliza para aplicar el algoritmo de encriptación SHA256.



```
Administrador: Símbolo del sistema

C:\>cd users

C:\Users>cd jpalm

C:\Users\jpalm>mkdir servicios-rest-node

C:\Users\jpalm>cd servicios-rest-node

C:\Users\jpalm\servicios-rest-node>npm init -y
Note to C:\Users\jpalm\servicios-rest-node\package.json:

{
  "name": "servicios-rest-node",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

C:\Users\jpalm\servicios-rest-node>npm install express body-parser crypto
npm warn deprecated crypto@1.0.1: This package is no longer supported. It's now a built-in Node module. If you've depended on crypto,
you should switch to the one that's built-in.

added 66 packages, and audited 67 packages in 3s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\jpalm\servicios-rest-node>
```

Figure 1: CREACIÓN DEL PROYECTO

### 3.2. Paso 2: Creación del Servidor

Posteriormente creé un archivo llamado `server.js`, donde implementé la lógica del servidor y los diferentes endpoints solicitados.

A continuación se muestra el código completo del servidor:

```
// Importación de librerías
const express = require('express');
const bodyParser = require('body-parser');
const crypto = require('crypto');

// Creación de la aplicación
const app = express();

// Configuración para recibir JSON
app.use(bodyParser.json());

// Definición del puerto
const PORT = 3000;
```

```
/* =====  
SERVICIO 1: mascaracteres  
===== */
```

```
app.post('/mascaracteres', (req, res) => {  
  
  const { cadena1, cadena2 } = req.body;  
  
  if (!cadena1 || !cadena2) {  
    return res.json({ success: false, error: "Faltan parámetros" });  
  }  
  
  let resultado = cadena1.length >= cadena2.length ? cadena1 : cadena2;  
  
  res.json({  
    success: true,  
    resultado: resultado  
  });  
});
```

```
/* =====  
SERVICIO 2: menoscaracteres  
===== */
```

```
app.post('/menoscaracteres', (req, res) => {  
  
  const { cadena1, cadena2 } = req.body;  
  
  if (!cadena1 || !cadena2) {  
    return res.json({ success: false, error: "Faltan parámetros" });  
  }  
  
  let resultado = cadena1.length <= cadena2.length ? cadena1 : cadena2;  
  
  res.json({  
    success: true,  
    resultado: resultado  
  });  
});
```

```
});

/* =====
SERVICIO 3: numcaracteres
===== */

app.post('/numcaracteres', (req, res) => {

  const { cadena } = req.body;

  if (!cadena) {
    return res.json({ success: false, error: "Falta la cadena" });
  }

  res.json({
    success: true,
    numeroCaracteres: cadena.length
  });
});

/* =====
SERVICIO 4: palindroma
===== */

app.post('/palindroma', (req, res) => {

  const { cadena } = req.body;

  if (!cadena) {
    return res.json({ success: false, error: "Falta la cadena" });
  }

  const limpia = cadena.replace(/\s+/g, '').toLowerCase();
  const invertida = limpia.split('').reverse().join('');

  res.json({
    success: true,
    resultado: limpia === invertida
  });
});
```

```
    });
  });

  /* =====
  SERVICIO 5: concat
  ===== */

  app.post('/concat', (req, res) => {

    const { cadena1, cadena2 } = req.body;

    if (!cadena1 || !cadena2) {
      return res.json({ success: false, error: "Faltan parámetros" });
    }

    res.json({
      success: true,
      resultado: cadena1 + cadena2
    });
  });

  /* =====
  SERVICIO 6: applysha256
  ===== */

  app.post('/applysha256', (req, res) => {

    const { cadena } = req.body;

    if (!cadena) {
      return res.json({ success: false, error: "Falta la cadena" });
    }

    const hash = crypto.createHash('sha256')
      .update(cadena)
      .digest('hex');

    res.json({
```

```
        success: true,
        original: cadena,
        encriptada: hash
    });
});

/* =====
SERVICIO 7: verifysha256
===== */

app.post('/verifysha256', (req, res) => {

    const { cadenaNormal, cadenaEncriptada } = req.body;

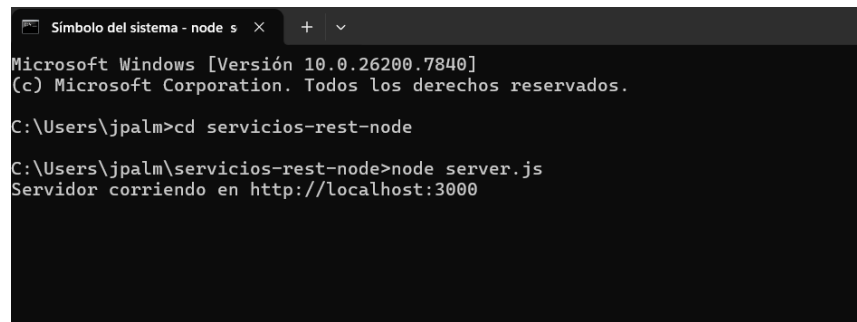
    if (!cadenaNormal || !cadenaEncriptada) {
        return res.json({ success: false, error: "Faltan parámetros" });
    }

    const hash = crypto.createHash('sha256')
        .update(cadenaNormal)
        .digest('hex');

    res.json({
        success: true,
        resultado: hash === cadenaEncriptada
    });
});

/* =====
INICIO DEL SERVIDOR
===== */

app.listen(PORT, () => {
    console.log('Servidor corriendo en http://localhost:${PORT}');
});
```



```
Símbolo del sistema - node s × + v
Microsoft Windows [Versión 10.0.26200.7840]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\jpalm>cd servicios-rest-node

C:\Users\jpalm\servicios-rest-node>node server.js
Servidor corriendo en http://localhost:3000
```

Figure 2: INICIO DEL SERVIDOR

### 3.3. Descripción General

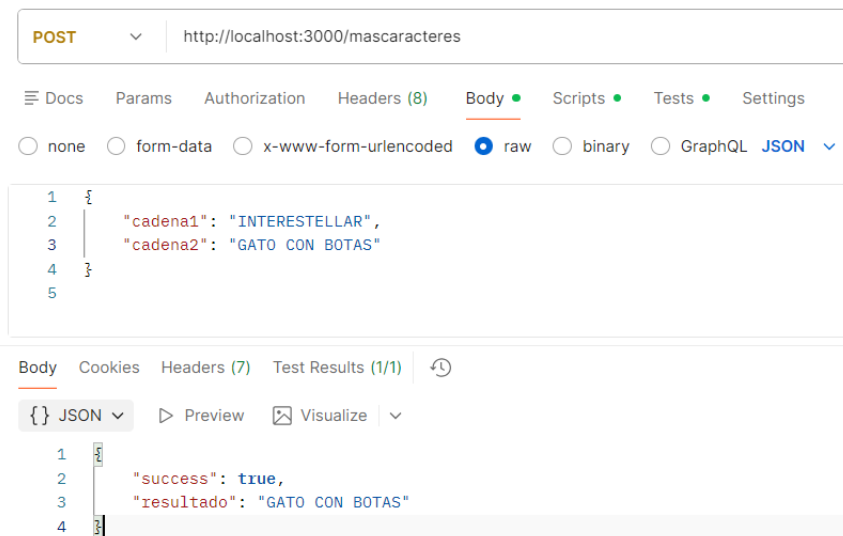
En este archivo configuré el servidor utilizando Express, establecí el puerto 3000 y desarrollé los siete servicios solicitados. Cada uno valida los parámetros recibidos y responde en formato JSON, incluyendo un campo `success` para indicar si la operación fue exitosa o un campo `error` en caso contrario.

Finalmente, inicié el servidor utilizando el método `app.listen()`, lo que permite que las peticiones realizadas desde Postman sean procesadas correctamente.

## 4. Implementación de los Servicios

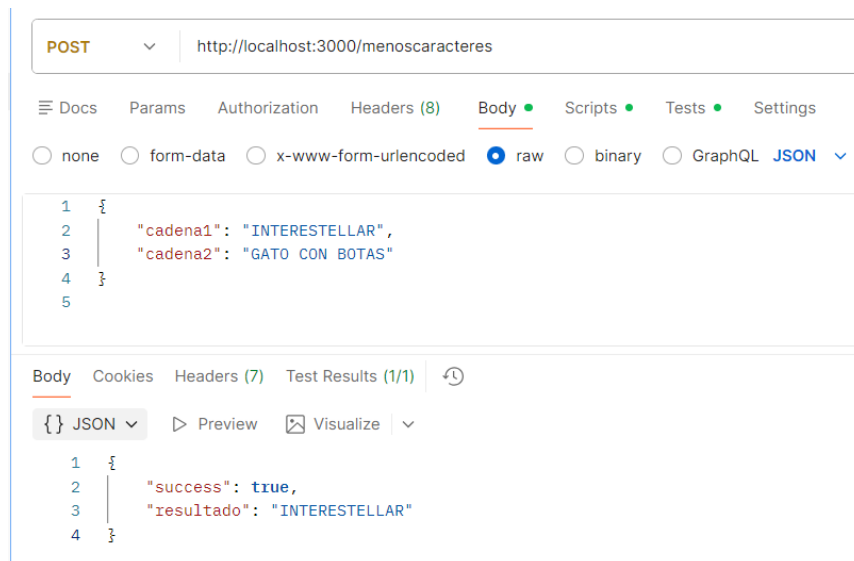
### 4.1. Servicio: mascaracteres

- Recibí dos cadenas en formato JSON.
- Validé que ambas existieran.
- Comparé sus longitudes.
- Si son iguales, regresé la primera.
- Respondí en formato JSON con `success`.



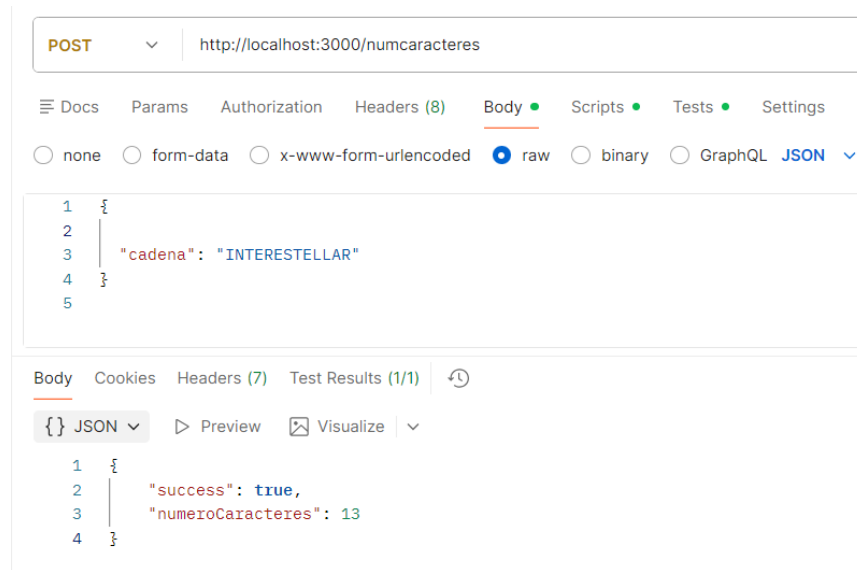
## 4.2. Servicio: menoscaracteres

Aquí hice lo mismo, pero ahora regresé la cadena más corta.



### 4.3. Servicio: numcaracteres

- Recibí una cadena.
- Validé que existiera.
- Regresé su longitud.



### 4.4. Servicio: palindroma

- Eliminé espacios.
- Convertí la cadena a minúsculas.
- La invertí.
- Comparé ambas.
- Regresé true o false.

POST ▼ http://localhost:3000/palindroma

Docs Params Authorization Headers (8) **Body** ● Scripts ● Tests ● Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {  
2   | "cadena": "anita lava la tina"  
3   | }  
4 }
```

Body Cookies Headers (7) Test Results (1/1) ↻

**{}** JSON ▼ ▶ Preview 🖼 Visualize ▼

```
1 {  
2   | "success": true,  
3   | "resultado": true  
4   | }  
5 }
```

## 4.5. Servicio: concat

Aquí concatené iniciando con el primer parámetro.

POST ▼ http://localhost:3000/concat

Docs Params Authorization Headers (8) **Body** ● Scripts ● Tests ● Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {  
2   | "cadena1": "Hola ",  
3   | "cadena2": "mundo"  
4   | }  
5 }
```

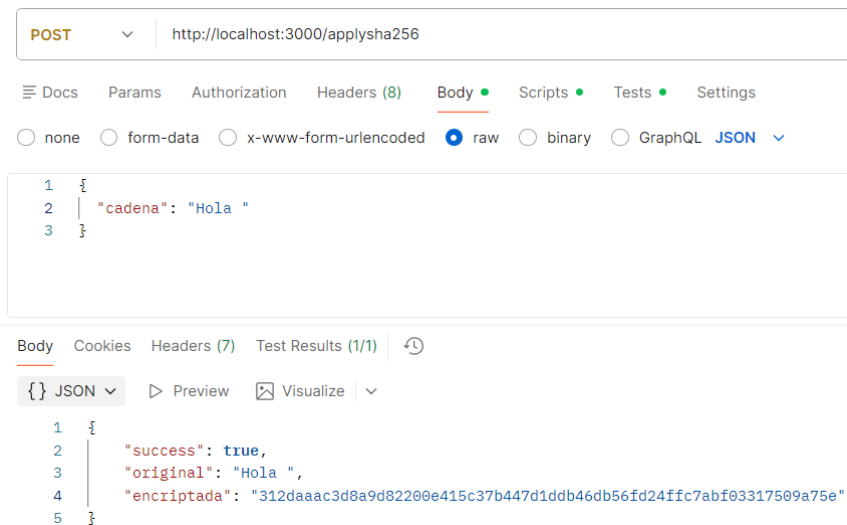
Body Cookies Headers (7) Test Results (1/1) ↻

**{}** JSON ▼ ▶ Preview 🖼 Visualize ▼

```
1 {  
2   | "success": true,  
3   | "resultado": "Hola mundo"  
4   | }  
5 }
```

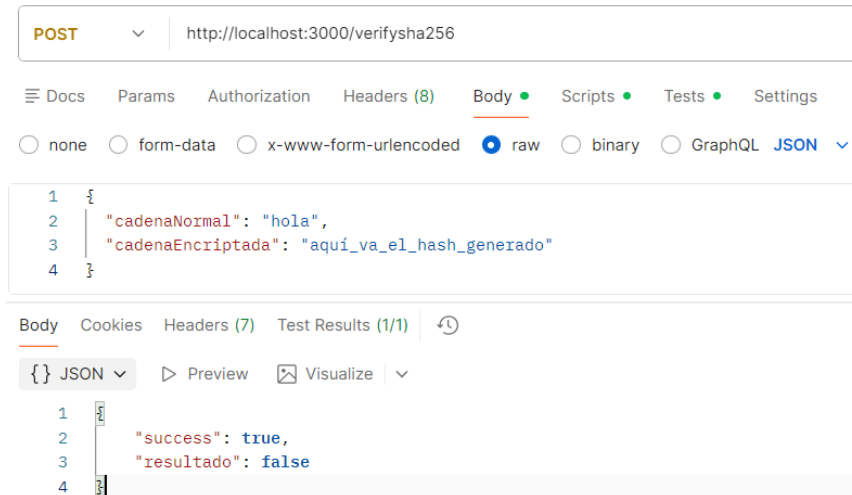
## 4.6. Servicio: applysha256

- Apliqué SHA256 usando crypto.
- Regresé la cadena original y la encriptada.



## 4.7. Servicio: verifysha256

- Encripté la cadena normal.
- La comparé con la encriptada.
- Regresé true o false.



## 5. Conclusiones

En esta práctica desarrollé una serie de servicios REST utilizando NodeJS y Express. Implementé validaciones en cada endpoint para asegurar que los parámetros fueran correctos y que todas las respuestas estuvieran en formato JSON. También apliqué el algoritmo SHA256 para el manejo de cadenas encriptadas y verifiqué su funcionamiento mediante pruebas en Postman. Finalmente, documenté el proyecto y lo subí a GitHub como evidencia del trabajo realizado.